

Protocolo BerryMed PM6750

cómo se comunica la app con el equipo

Contenido

1. Mapa de archivos
2. Configuración de la aplicación
 - 2.1 Conexión con el backend
 - 2.2 Conexión con el equipo
 - 2.3 Configuración del equipo (PM6750)
3. Transporte
4. Formato de trama
5. Comandos app → equipo
 - 5.1 Secuencia de arranque
 - 5.2 Medición de presión (bajo demanda)
 - 5.3 Tabla completa de comandos
6. Paquetes equipo → app
 - 6.1 `0x01` — onda de ECG
 - 6.2 `0x02` — parámetros de ECG
 - 6.3 `0x03` — NIBP
 - 6.4 `0x04` — SpO₂
 - 6.5 `0x05` — temperatura
 - 6.6 `0x30` — marca de latido (QRS)
7. Payload que se postea
8. Detalle de implementación: guardar ≠ notificar
9. Auditar el stream real
10. Pendientes conocidos

Protocolo BerryMed PM6750 — cómo se comunica la app con el equipo

Documentación técnica de la comunicación entre esta aplicación y el módulo multiparamétrico **BerryMed PM6750**: cómo se conecta, qué le manda, qué recibe y cómo eso termina en el payload que se postea a la API.

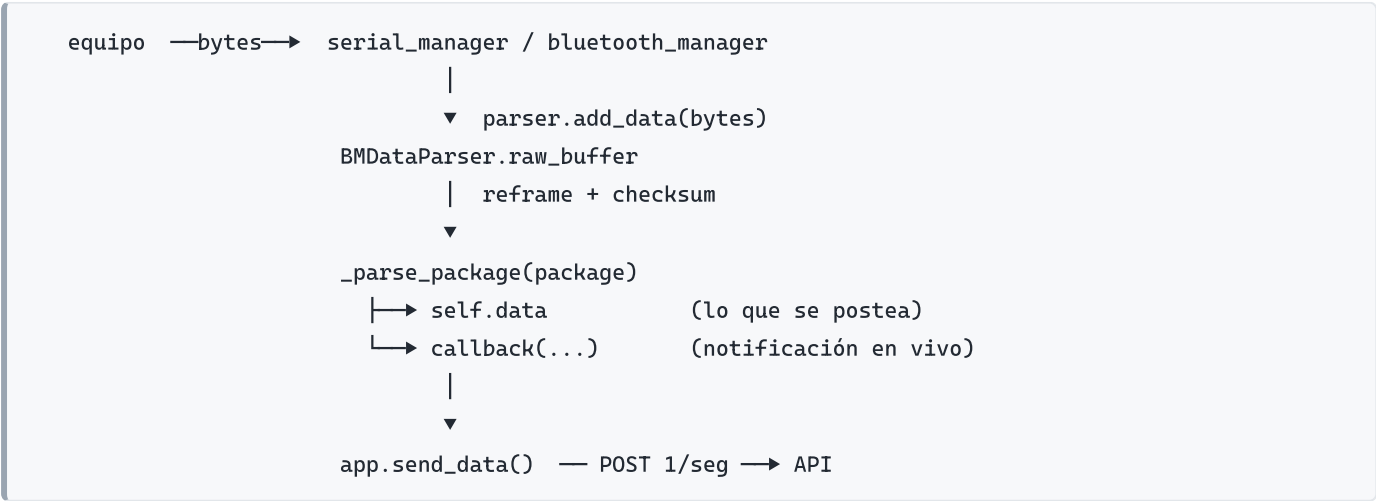
Fuentes: manual técnico del equipo (docs/manual_berry.pdf) y verificación sobre capturas reales del stream. Cuando algo se comprobó midiendo y no sólo leyendo el manual, está marcado como **(medido)**.

Para el detalle específico de ECG y derivaciones, ver ecg.md .

1. Mapa de archivos

Archivo	Rol
src/pm6750_protocol.py	Única fuente de verdad de los comandos PC→equipo. Constantes, checksum, armado de tramas y secuencia de arranque.
src/serial_manager.py	Transporte USB/RS232 (PM6750USBReader). Abre el puerto, manda el arranque, lee en un hilo.
src/bluetooth_manager.py	Transporte BLE (BMPatientMonitor). Descubre el equipo, se suscribe a notificaciones, manda el arranque.
src/data_parser.py	Desarma las tramas que llegan (BMDDataParser) y arma self.data , que es el payload.
app.py	Orquesta: elige transporte, registra callbacks y postea a la API una vez por segundo.
config.py	Lee credentials.json y expone la configuración ya normalizada.
configure.py	Punto de entrada del asistente que escribe credentials.json (§2).
src/config_schema.py	Definición de las claves de configuración, su validación y el guardado.
src/configure_gui.py	Asistente gráfico (tkinter), armado a partir del schema.
src/thermometer_reader.py	Termómetro externo por serie, independiente del PM6750.

Flujo general:



2. Configuración de la aplicación

Toda la configuración vive en un único archivo JSON, que crea el asistente `configure.py` y lee `config.py` :

Sistema	Ubicación
Windows	<code>%APPDATA%\BerryMed Monitor\credentials.json</code>
Otros	<code>~/.berrymed/credentials.json</code>

`get_app_data_path()` resuelve en este orden: la variable de entorno `BERRYMED_CONFIG_DIR` si está definida (útil para pruebas), después la ruta del sistema operativo, y si se está corriendo **bajo WSL sin config propia**, la configuración de Windows. Eso último permite correr la app y el configurador desde WSL contra la misma configuración que usa el ejecutable, en vez de contra un archivo vacío.

⚠ Consecuencia: si se guarda desde WSL, **se escribe el archivo real de Windows**. El asistente muestra siempre la ruta exacta sobre la que va a trabajar, arriba de todo.

```
python configure.py          # asistente gráfico (cae a texto si no hay display)
python configure.py --cli    # fuerza el asistente de texto
```

Como ejecutable. `berry-configure.exe` se compila en modo ventana (`console=False` en `berry-configure.spec`): al abrirlo con doble clic muestra sólo la ventana, sin consola negra de fondo. En ese modo el proceso **no tiene `stdout`**, así que el asistente de texto no puede correr; para usarlo hay que lanzarlo desde una terminal, y ahí el exe se engancha a esa consola:

```
berry-configure.exe          → ventana
berry-configure.exe --cli    → asistente de texto, en la terminal desde donde se lo llamó
```

Si abierto con doble clic la ventana no pudiera abrirse, se avisa con un cuadro de diálogo nativo de Windows explicando cómo usar `--cli` — nunca queda un proceso mudo ni un crash por escribir en una consola inexistente.

Si el archivo no existe o no se puede leer, `get_config()` devuelve `None` y la app aborta al arrancar con `Error: No se encontraron credenciales`.

El asistente carga la configuración existente y la ofrece como valor de partida, así reconfigurar es editar y no volver a cargar todo. Al guardar conserva las claves que no pregunta (§2.3) y cualquier clave desconocida que haya en el archivo.

Los campos de los dos asistentes salen de `src/config_schema.py`, que es la única fuente de verdad: agregar una clave ahí la hace aparecer en ambos, con su validación. Las opciones cerradas (modo de ECG, ganancias, tipo de conexión) son desplegables en la interfaz gráfica y se revalidan en el texto, así que no se puede guardar un valor inválido.

2.1 Conexión con el backend

Clave	Default	Para qué sirve
<code>API_URL</code>	—	Endpoint donde se postean los datos, 1 vez por segundo
<code>API_USERNAME</code>	—	Usuario de la auth básica del POST
<code>API_PASSWORD</code>	—	Contraseña de la auth básica
<code>PUSHER_KEY</code>	—	App key de Pusher, para recibir órdenes
<code>PUSHER_CLUSTER</code>	<code>us2</code>	Cluster de Pusher
<code>PUBLIC_CHANNEL</code>	—	Canal al que se suscribe la app
<code>START_EVENT_NAME</code>	<code>start-monitoring</code>	Evento que arranca el envío de datos
<code>STOP_EVENT_NAME</code>	<code>stop-monitoring</code>	Evento que corta el envío y resetea el parser
<code>SSL_CERT_FILE_PATH</code>	—	Bundle de certificados; se exporta como <code>SSL_CERT_FILE</code>
<code>TOTEM_ID</code>	—	Identificador del tótem

La app **no envía nada hasta recibir el evento de arranque**: `is_sending_data` arranca en `False` y el loop de `send_data()` duerme hasta que llega.

⚠ Dos detalles a tener en cuenta: - El evento de presión arterial, `start-blood-pressure`, está **hardcodeado** en `app.py` — a diferencia de arranque y parada, no se puede renombrar desde la configuración. - `TOTEM_ID` se lee y se expone en la config, pero **hoy no lo usa ningún módulo**: no viaja en el payload ni en los headers del POST.

2.2 Conexión con el equipo

Clave	Default	Valores
<code>DEVICE_CONNECTION</code>	<code>bt</code>	<code>bt</code> (Bluetooth LE) o <code>usb</code> (serie/RS232)
<code>DEVICE_PORT</code>	<code>COM3</code>	Puerto serie; sólo se usa si <code>DEVICE_CONNECTION=usb</code>
<code>THERMOMETER_ENABLED</code>	<code>false</code>	<code>true</code> / <code>false</code> . Termómetro externo, separado del PM6750
<code>THERMOMETER_PORT</code>	<code>COM5</code> (Windows) / <code>/dev/ttyACM0</code>	Puerto serie del termómetro
<code>THERMOMETER_BAUD</code>	<code>115200</code>	Baudios del termómetro
<code>THERMOMETER_RECONNECT_SECONDS</code>	<code>2.0</code>	Espera entre reintentos de reconexión

El termómetro es un dispositivo **aparte**: lo lee `ThermometerReader` en su propio hilo y su valor pisa `vitalSigns.temperature` justo antes de postear.

2.3 Configuración del equipo (PM6750)

Clave	Default	Valores	¿La pregunta <code>configure.py</code> ?
<code>ECG_MODE</code>	<code>monitor</code>	<code>operacion</code> , <code>monitor</code> , <code>diagnostico</code>	sí
<code>ECG_GAIN</code>	<code>x1</code>	<code>x0.25</code> , <code>x0.5</code> , <code>x1</code> , <code>x2</code>	sí
<code>RESP_GAIN</code>	<code>x1</code>	idem	sí
<code>NIBP_MODE</code>	<code>adulto</code>	<code>adulto</code> , <code>nino</code> , <code>neonato</code>	sí
<code>NIBP_TARGET_PRESSURE_MMHG</code>	<i>(sin definir)</i>	entero, dentro del rango del modo	no — avanzada

Estos valores se traducen a comandos del protocolo y se mandan al conectar (§5.1) o al pedir una medición de presión (§5.2). Los defaults coinciden con los valores de fábrica del equipo (medido: reporta ganancia x1 y modo monitor), así que mandarlos explícitamente deja la configuración **determinista** en vez de depender del estado interno del módulo. Un valor inválido no rompe nada: se avisa por consola y se cae al default.

Cuándo tocarlos:

- **La onda satura** (valores pegados a 0 y 250): bajar `ECG_GAIN` a `x0.5` o `x0.25`. En la captura de referencia el 98,1 % de las muestras estaban saturadas con ganancia x1, y el equipo reportaba `LeadOff: False` — o sea que no era electrodo suelto sino el amplificador saturando.
- **Uso diagnóstico:** `ECG_MODE=diagnostico` (0,05–100 Hz) conserva el segmento ST, que el modo monitor filtra.

Qué es `NIBP_TARGET_PRESSURE_MMHG`. Es hasta cuántos mmHg infla el manguito al empezar a medir la presión. El método es oscilométrico: el equipo infla por encima de la sistólica y después desinfla midiendo las oscilaciones, así que ese valor es el *techo de inflado*, no un resultado.

Por defecto el equipo usa 150 mmHg en adulto, 100 en niño y 70 en neonato, y con eso alcanza en la enorme mayoría de los casos. Sólo tiene sentido tocarlo si el paciente es muy hipertenso y la medición falla por quedarse corta de inflado (hay que subirlo), o si se quiere inflar menos por comodidad en alguien con presión baja. Rangos válidos: adulto 40–300, niño 40–210, neonato 40–140; fuera de rango se ignora con un warning.

Queda fuera del asistente a propósito: es una decisión clínica, no de instalación. Si hace falta, se agrega a mano al `credentials.json` — el asistente lo conserva al reconfigurar:

```
{ "NIBP_TARGET_PRESSURE_MMHG": 180 }
```

⚠ Hoy es un valor **fijo por instalación**, y eso no encaja del todo: el `configure.py` se corre una vez y la presión adecuada depende del paciente. Lo natural sería que viajara en la orden de medición (evento `start-blood-pressure`) en vez de estar en la configuración. Pendiente de definir con el backend — ver §10.

3. Transporte

	USB / RS232	Bluetooth LE
Clase	PM6750USBReader	BMPatientMonitor
Se elige con	DEVICE_CONNECTION="usb"	DEVICE_CONNECTION="bt" (default)
Parámetros	115200 baudios, 8N1, sin paridad	nombre del dispositivo: BerryMed
Lectura	hilo daemon con <code>serial.read(256)</code>	<code>start_notify</code> sobre <code>CHAR_RECEIVE_UUID</code>
Escritura	<code>serial.write</code>	<code>write_gatt_char</code> sobre <code>CHAR_SEND_UUID</code>

UUIDs BLE:

```
SERVICE 49535343-fe7d-4ae5-8fa9-9fafd205e455
RECEIVE 49535343-1e4d-4bd9-ba61-23c647249616 (notificaciones: equipo → app)
SEND 49535343-8841-43f4-a8d4-ecbe34729bb3 (comandos: app → equipo)
```

Nota de reframing. `serial_manager._loop` ya corta las tramas antes de pasárselas al parser, y el parser vuelve a buscar la cabecera por su cuenta. Es redundante pero inofensivo: `add_data` acumula en `raw_buffer` y resincroniza solo. La vía BLE entrega los chunks crudos y deja todo el trabajo al parser.

4. Formato de trama

Es el mismo en los dos sentidos:

```
byte:  0      1      2      3      4      5      ...      -1
       0x55  0xAA  length type/A1 A2  A3  ...  An  checksum
```

- `0x55 0xAA` — cabecera fija.
- `length (N)` — $N = n + 2$, donde `n` es la cantidad de bytes de datos (`A1 . . . An`). El fin de la trama está en `inicio + N + 2`.
- `checksum` — $\sim(N + A1 + A2 + \dots + An) \& 0xFF$.

En el código: `pm6750_protocol.checksum()` para armar, `BMDDataParser._check_sum()` para validar. Son la misma fórmula desde los dos lados.

Ejemplo (onda de ECG, del manual pág. 12):

```
55 AA 0A 01 80 80 80 80 80 80 80 74
| | | | └── 7 derivaciones ───┘ └─ checksum
| | | └─ type = 0x01
| | └─ N = 10 → 8 bytes de datos (type + 7) + checksum
└─ cabecera
```

5. Comandos app → equipo

El equipo no transmite nada hasta recibir los comandos de habilitación.

Todos tienen la forma `55 AA 04 A1 A2 SUM`, con `A1` = comando y `A2` = valor. Se arman con `pm6750_protocol.build_command(a1, a2)`.

5.1 Secuencia de arranque

`build_startup_commands(config)` devuelve, en este orden — primero configurar, después abrir los streams, para que las muestras salgan ya configuradas:

#	Comando	A1	A2 (default)	Qué hace
1	Ganancia de ECG	0x07	0x03 (x1)	Configurable con <code>ECG_GAIN</code>
2	Modo de ECG	0x08	0x02 (monitor)	Configurable con <code>ECG_MODE</code>
3	Ganancia de respiración	0x0F	0x03 (x1)	Configurable con <code>RESP_GAIN</code>
4	Habilitar ECG	0x01	0x01	Habilita onda 0x01, params 0x02 y picos 0x30
5	Habilitar SpO ₂	0x03	0x01	
6	Habilitar temperatura	0x04	0x01	
7	Habilitar onda SpO ₂	0xFE	0x01	
8	Habilitar onda de respiración	0xFF	0x01	

En el arranque no va ningún comando de NIBP. Ni el de medición (0x02) ni los de configuración (0x09 modo, 0x0A presión objetivo). Al conectar no sabemos si el paciente tiene puesto el manguito, así que toda la presión se maneja bajo demanda (§5.2).

Las dos vías de conexión mandan exactamente esta misma secuencia. Por BLE se intercala una pausa de 50 ms entre comandos, porque el módulo pierde escrituras si llegan pegadas, y es best-effort: si falla se avisa por `status_callback` pero no se corta la conexión.

5.2 Medición de presión (bajo demanda)

`build_nibp_commands(config)` se manda sólo cuando se pide la medición — la dispara el evento Pusher `start-blood-pressure` → `handle_blood_pressure_event` → `monitor.start_nibp()`:

#	Comando	A1	A2	Qué hace
1	Modo NIBP	0x09	0x01 (adulto)	Configurable con <code>NIBP_MODE</code>
2	Presión objetivo	0x0A	(omitido)	Sólo si se define <code>NIBP_TARGET_PRESSURE_MMHG</code>
3	Arrancar medición	0x02	0x01	Infla el manguito

La configuración va acá y no en la conexión por dos razones: no inflar nada hasta que lo pidan, y porque el manual (pág. 7) pide setear la presión objetivo **inmediatamente antes** de cada medición — si no, el equipo usa su default.

El cierre lo maneja `_nibp_done`: al recibir un 0x03 con resultado terminal libera el flag `nibp_running` y manda un STOP explícito (0x02 0x00). Hay además un watchdog de 90 s por si nunca llega el cierre.

5.3 Tabla completa de comandos

A1	Comando	Valores de A2
0x01	Control de ECG	0x00 sin salida, 0x01 con salida
0x02	Control de NIBP	0x00 parar medición, 0x01 arrancar
0x03	Control de SpO ₂	0x00 / 0x01
0x04	Control de temperatura	0x00 / 0x01
0x07	Ganancia de ECG	0x01 x0.25, 0x02 x0.5, 0x03 x1, 0x04 x2
0x08	Modo de ECG	0x01 operación (1–25 Hz), 0x02 monitor (0,5–75 Hz), 0x03 diagnóstico (0,05–100 Hz)
0x09	Modo NIBP	0x01 adulto, 0x02 niño, 0x03 neonato
0x0A	Presión objetivo NIBP	mmHg ÷ 2. Adulto 40–300 (def. 150), niño 40–210 (def. 100), neonato 40–140 (def. 70)
0x0B	Test de presión estática	⚠ no usar con personas
0x0C	Calibración de presión	⚠ sólo fábrica
0x0D / 0x0E	Calibración de Temp1 / Temp2	⚠ sólo fábrica
0x0F	Ganancia de respiración	igual que 0x07
0x10	Test de fuga NIBP	⚠ sólo test
0xFC	Leer versión de software	A2 sin uso
0xFD	Leer versión de hardware	A2 sin uso
0xFE	Salida de onda de SpO ₂	0x00 / 0x01
0xFF	Salida de onda de respiración	0x00 / 0x01

Los comandos marcados ⚠ no los usa la app y no deberían usarse sin motivo: 0x0B – 0x0E y 0x10 son de calibración/test de fábrica.

6. Paquetes equipo → app

type es el byte 3 de la trama. El parser los mapea a nombres de callback en `BMDDataParser.callbacks`.

type	Nombre interno	Contenido	Manual	Medido
0x01	on_ecg_waveform_received	7 derivaciones de ECG	250 /seg	250,7 /seg
0x02	on_ecg_params_received	Estado, HR, FR, ST, código HRV	1 /seg	1 /seg
0x03	on_nibp_params_received	Estado, presión del manguito, SBP, MBP, DBP	2 /seg	(sin datos)
0x04	on_spo2_params_received	Estado, SpO ₂ , pulso	1 /seg	0,93 /seg
0x05	on_temp_params_received	Estado, T1, T2	1 /seg	0,93 /seg
0x30	on_ecg_peak_received	Marca de latido (QRS)	no documentado	1 por latido
0x31	on_spo2_peak_received	Marca de pulso	no documentado	(sin datos)
0xFE	on_spo2_waveform_received	Onda de SpO ₂	40 /seg	50 /seg
0xFF	on_resp_waveform_received	Onda de respiración	50 /seg	50 /seg

Dos discrepancias con el manual, ambas verificadas sobre la captura:

- 0x30 y 0x31 no aparecen en el manual. La tabla *Module→PC* de la pág. 10 no los lista. Existen igual: el equipo los manda y los nombres vienen del SDK de BerryMed. Que 0x30 sea la marca de QRS está confirmado midiendo (§6.6), no leyendo. 0x31 no apareció en la captura (no había dedo en el sensor).
- La onda de SpO₂ llega a 50 /seg, no a 40 como dice el manual.

La columna "medido" sale de una captura de 30 s con 10.618 tramas y 0 checksums inválidos.

6.1 0x01 — onda de ECG

7 bytes, una derivación cada uno, del mismo instante (no son 7 muestras consecutivas). Orden confirmado:

I, II, III, aVR, aVL, aVF, V. Rango 0–250, línea de base 128. Detalle completo y evidencia en `ecg.md` §4.1.

6.2 0x02 — parámetros de ECG

byte	Campo	Destino en el payload
[4]	Estado del ECG (bits)	<code>ecgInfo.leadOff</code> , <code>.weakSignal</code> , <code>.gain</code> , <code>.mode</code>
[5]	Frecuencia cardíaca (0–250)	<code>vitalSigns.heartRate</code>
[6]	Frecuencia respiratoria (0–250)	<code>vitalSigns.respRate</code>
[7]	ST (signed char, –100...+100 = –1...+1 mV)	<code>ecgInfo.st</code> (en mV)
[8]	Código HRV / arritmia	<code>ecgInfo.hrv</code>

Bits del byte de estado:

bits	Significado
BIT0	Señal débil (1 = débil)
BIT1	Electrodo suelto (1 = lead off)
BIT3~2	Ganancia: 00 x0.25, 01 x0.5, 10 x1, 11 x2
BIT5~4	Modo: 00 operación, 01 monitor, 10 diagnóstico

Códigos HRV (`BMDDataParser.HRV_CODES`): 0x00 analizando, 0x01 normal, 0x02 paro, 0x03 fibrilación ventricular, 0x04 R sobre T, 0x05 – 0x07 extrasístoles ventriculares, 0x08 bigeminismo, 0x09 trigeminismo, 0x0A taquicardia, 0x0B bradicardia, 0x0C latido perdido.

ST y HRV se leen sólo si la trama los trae (`payload_len >= 4` / `>= 5`): hay firmwares que mandan el paquete más corto.

6.3 0x03 — NIBP

byte	Campo
[4]	Estado: BIT1~0 modo (adulto/niño/neonato), BIT5~2 estado del test
[5]	Presión del manguito — $\text{valor} \times 2 = \text{mmHg}$
[6]	Sistólica (mmHg)
[7]	Presión arterial media (mmHg)
[8]	Diastólica (mmHg)

Estados del test (BIT5~2): 0000 terminado, 0001 midiendo, 0010 detenido, 0011 presión > 300 mmHg, 0100 manguito flojo, 0101 demoró demasiado, 0110 error, 0111 interferencia, 1000 sin resultado, 1001 inicializando, 1010 inicialización terminada.

`serial_manager._nibp_done` considera terminales los resultados 0x0, 0x2, 0x4 y 0x5, libera el flag `nibp_running` y manda un STOP explícito.

⚠ El parser guarda **sistólica y diastólica** ([6] y [8]) pero **descarta la presión arterial media** ([7]), que el equipo sí manda.

6.4 0x04 — SpO₂

[4] estado (0x00 normal, 0x01 sensor suelto, 0x02 sin dedo, 0x03 buscando pulso, 0x04 búsqueda demasiado larga), [5] SpO₂ 0–100 (127 = error), [6] pulso 0–250 (255 = error).

El pulso vive en `vitalSigns.spo2Pulse`, con formato "SpO2/pulso":

SpO ₂	Pulso	spo2Pulse
98	72	"98/72"
127 (error)	72	"-/72"
98	255 (error)	"98/-"
127 (error)	255 (error)	"- - /- -"

Cada valor se valida por separado contra su propio rango, así un SpO₂ con error no invalida un pulso bueno. El sentinela "- - /- -" se reserva para cuando fallan los dos: `_is_valid_data()` lo usa para decidir si hay algo que postear.

6.5 0x05 — temperatura

[4] estado (0x00 normal, 0x01 T1 suelto, 0x02 T2 suelto, 0x03 ambos), [5] parte entera, [6] decimal.

Temperatura real = entero + decimal/10.

⚠ El parser hace `(package[5] * 10 + package[6]) / 10.0`, que es la misma cuenta, pero **sólo lee T1**: los bytes de T2 ([7], [8]) se ignoran.

6.6 0x30 — marca de latido (QRS)

Este paquete no está en el manual. El equipo lo manda igual y el nombre viene del SDK de BerryMed; lo que sigue se verificó midiendo.

Sin bytes de datos: el paquete en sí es el evento. El equipo manda **uno por cada latido detectado**. Evidencia — usando los paquetes de onda como reloj (~251 Hz), el intervalo mediano entre 0x30 es de **718 ms ≈ 84 lpm**, contra los **86 lpm** que reportaba el 0x02 en la misma captura. Los huecos de 2,1 a 3,5 s que aparecen intercalados coinciden con los tramos saturados: ahí el detector del equipo perdió latidos.

Se guarda en `data.ecgPeaks` como la posición del latido **dentro de la ventana de ECG del mismo POST**, para poder marcarlo sobre la onda:

```
{"sample": 192, "ms": 768.0}
```

`sample` es el índice dentro de `data.ecg.*`; `ms` es ese índice llevado a tiempo con `ECG_SAMPLE_RATE_HZ` (250 Hz nominal).

Sirve para marcar latidos sobre la tira y para calcular variabilidad R-R, que el 0x02 no permite porque ahí el HR ya viene promediado y a 1 Hz.

7. Payload que se postea

`app.send_data()` postea una vez por segundo a `API_URL` con auth básica:

```
{
  "timestamp": 1754400000000,
  "data": {
    "ecg": {                                // las 7 derivaciones, ~251 valores/seg c/u
      "I": [...], "II": [...], "III": [...], "aVR": [...],
      "aVL": [...], "aVF": [...], "V": [...]
    },
    "ecgPeaks": [                          // latidos detectados en esta ventana
      {"sample": 192, "ms": 768.0}
    ],
    "ecgInfo": {                           // estado y diagnóstico del ECG
      "leadOff": false, "weakSignal": false,
      "gain": "x1", "mode": "monitor",
      "st": 0.06, "hrv": "normal"
    },
    "spo2": [...],                        // ~50 valores/seg
    "resp": [...],                        // ~50 valores/seg
    "vitalSigns": {
      "heartRate": "85", "respRate": "16",
      "spo2Pulse": "98/72", "nibp": "120/80", "temperature": "36.6"
    }
  }
}
```

Ventanas de 1 segundo. Todos los arrays se vacían y se rearmen cada segundo (`_wave_last` por onda). `ecg` y `ecgPeaks` comparten la misma ventana — se vacían juntos, o los índices de los picos apuntarían a la ventana anterior (`_roll_ecg_window`).

Sin dato se representa como `"-"` o `"- -"` en `vitalSigns`, y como `null` en `ecgInfo`. `_is_valid_data()` no postea si no hay ningún signo vital ni ninguna onda.

Tope defensivo: `max_waveform_points = 2500` por ventana y `max_peaks_per_window = 100`, para que un stream corrupto no infle el payload.

8. Detalle de implementación: guardar ≠ notificar

Cada tipo de paquete tiene dos efectos **independientes**:

1. **Guardar** en `self.data` → es lo que viaja en el POST.
2. **Notificar** al callback registrado → para reaccionar en vivo.

Los dos están desacoplados a propósito: si nadie registró el callback, `_parse_package` usa un no-op y el guardado sigue su curso igual. **El payload no depende de que alguien esté escuchando** — registrar un callback es opcional y sirve sólo para reaccionar en tiempo real, nunca para que el dato llegue a la API.

9. Auditar el stream real

```
# capturar 30 s del equipo y guardar el crudo
python tools/ecg_audit.py --port COM6 --seconds 30 --record captura.bin

# reanalizar una captura guardada, sin el equipo
python tools/ecg_audit.py --file captura.bin

# modo pasivo: escuchar sin mandar ningún comando
python tools/ecg_audit.py --port COM6 --passive
```

Nota operativa (WSL): el equipo es un STM32 VCP. Si se usó `usbip`, hacer `usbipd unbind` y reconectar el USB antes de usarlo desde Windows, o el puerto queda mudo (0 bytes).

10. Pendientes conocidos

- **Presión objetivo de NIBP por medición, no por instalación.** Hoy `NIBP_TARGET_PRESSURE_MMHG` se fija una sola vez en el `configure.py`, pero la presión de inflado adecuada depende del paciente, no del tótem. Lo natural sería que viajara en la propia orden (`start-blood-pressure`) y que `start_nibp()` la reciba como parámetro — el protocolo ya lo permite, porque el comando `0x0A` se manda igual justo antes de cada medición (§5.2). Falta definir con el backend si el evento puede transportar ese dato.
- **Presión arterial media (MBP)** del `0x03` — el equipo la manda, el parser la descarta (§6.3).
- **Segundo canal de temperatura (T2)** del `0x05` — se ignora (§6.5).
- `0x31` (marca de pulso de SpO₂) — llega y dispara el callback, pero no se guarda en el payload, **por decisión**: la frecuencia de pulso ya va en `vitalSigns.spo2Pulse` y no hace falta un campo aparte. Se guarda el equivalente de ECG (`0x30`) porque ahí sí aporta algo que no está en ningún otro lado: el timing latido a latido para marcar la onda y calcular R-R.
- **Versiones de firmware** (`0xFC` / `0xFD`) — nunca se consultan; servirían para registrar con qué firmware se tomó cada medición.

- **Captura de referencia saturada** — repetir con ganancia más baja para tener una referencia limpia (§2.3).